# A Self-Hosted Zero-Knowledge Proof of Possession, Instantiated on the Divisor Lattice of the Monster

Formalised in Lean 4 / Mathlib

**Abstract**

We report a fully machine-checked zero-knowledge proof of possession (ZKPoP) whose statement, parameters and verifier all live inside a single formal development, and we instantiate it on an arithmetic result proved in the same development: the amicable pairs hidden in the divisor lattices of the sporadic simple groups. The arithmetic layer proves that among the 424 488 960 divisors of the Monster's order there are exactly three amicable pairs with smaller member below $10^7$, namely $(220, 284)$, $(67095, 71145)$ and $(522405, 525915)$, of which the last two are odd, and that $J_4$ carries the pair $(1184, 1210)$. The cryptographic layer is Schnorr's $\Sigma$-protocol together with Pedersen commitments, CDS OR-composition and bit-decomposition range proofs — all textbook constructions, reproduced verbatim; the group is $\mathbb{Z}/(2^{521}-1)$, whose modulus is proved prime by the Lucas–Lehmer test *in the same kernel*. Our contribution is deliberately not new mathematics: it is the observation, made precise and mechanically checked, that once the number theory, the group generation, the protocol proofs and the verifier are all objects of one kernel, the resulting zero-knowledge system is *self-hosted* — it needs no trusted setup, no external parameter generator, no unformalised soundness argument, and it can run on its own description. We state and prove a single theorem, `self_hosted_zk_system`, that packages these five conditions, and we give an explicit self-referential instance: the system proves possession of the digest of its own verifier's code. Total new formal content: one file of 236 lines resting on 819 lines of previously verified protocol code; new mathematics: none.

## Contents

# 1 Introduction

## 1.1 The point of the exercise

A zero-knowledge proof system, in deployment, is a stack of heterogeneous trust. The statement being proved is a mathematical assertion, usually informal or at best specified in a circuit language. The group parameters come from a setup ceremony or from a standards document. The soundness argument is a pen-and-paper reduction. The verifier is a piece of software whose relation to the pen-and-paper protocol is a matter of programming discipline. Each of these four layers is trusted separately, and each has been the site of real failures.

This paper takes an existing, entirely machine-checked piece of number theory and observes that when a zero-knowledge proof of possession is built *on top of the same kernel that checked the number theory*, the four layers collapse into one. We call the result a *self-hosted* zero-knowledge system, by direct analogy with a self-hosting compiler: a compiler is self-hosting when it compiles its own source; a zero-knowledge system is self-hosted when the same proof-checking kernel supplies

1. the *statement* that is proved in zero knowledge,
2. the *group*, via a primality theorem rather than a ceremony,
3. the *soundness, completeness and zero-knowledge* arguments, as theorems,
4. the *verifier*, as an executable function proved equal to its specification, and
5. an object encoding the system's own description, on which the system can be run.

## 1.2 Design constraint: reveal as little new mathematics as possible

The constraint we set ourselves — and the reason the paper is short on novelty by design — is that *every* mathematical and cryptographic ingredient should be a known one, used unchanged. Schnorr's protocol (1989/1991), Pedersen's commitment scheme (1991), Cramer–Damgård–Schoenmakers OR-composition (1994), bit-decomposition range proofs, Fiat–Shamir (1986), the Lucas–Lehmer primality test, and the multiplicativity of $\sigma_1$ are all classical. The arithmetic side, likewise, reuses the classical amicable pairs of Pythagoras and Euler rather than manufacturing new ones. What is new here is not a theorem but a *composition*: the observation that these standard pieces, once all inside one kernel, satisfy the five conditions above simultaneously, and the mechanical proof that they do.

This constraint has a second, cryptographic reading, and the two coincide pleasantly. A zero-knowledge protocol is supposed to reveal as little as possible about the witness; a minimal-novelty paper is supposed to reveal as little as possible that is new. Our system reveals, of the arithmetic witness, exactly one public equation — and of the mathematics, exactly one new definition (Definition 4.1).

### 1.3 Contributions

- A precise definition of a self-hosted zero-knowledge proof of possession (§4), and a single Lean theorem, `ZKPoP.SelfHost.self_hosted_zk_system`, certifying that our instance meets it.
- An executable verifier `verifyB` together with a proof that it decides the specification `ZKPoP.verify` (§4.2) — the implementation/specification gap, closed inside the kernel.
- A self-referential object: the digest of the verifier's own code is a legitimate secret of the system, and possession of it is proved, extracted and simulated exactly like any other (§4.4).
- A worked application in which the arithmetic result of the host development — the amicable pairs among the divisors of $|\mathbb{M}|$ and $|J_4|$ — is certified in zero knowledge on public keys alone (§5).
- An explicit trust ledger (§7) recording exactly what is and is not claimed, including the two places where the concrete instance is a faithful arithmetic model rather than a deployment.

### 1.4 What we do not claim

We do not claim a new cryptographic primitive, a new hardness assumption, a new efficiency result, or a new theorem of number theory. We do not claim that the concrete instance is secure to deploy: discrete logarithms in the representation $\mathbb{Z}/p$ are easy, and §7 says so precisely. Above all, we do not claim any form of self-verification in the Gödelian sense: a kernel cannot prove its own consistency, and nothing here tries to. Self-hosting is a statement about *provenance of components*, not about self-justification; §4.6 makes the distinction explicit.

## 2 The arithmetic layer: what is being proved in zero knowledge

### 2.1 The divisor lattice of the Monster

Write $\sigma_1(n) = \sum_{d \mid n} d$ and call $m \neq n$ *amicable* when $\sigma_1(m) = \sigma_1(n) = m + n$. The host development starts from the order of the Monster,

$$\begin{aligned} |\mathbb{M}| &= 2^{46}\,3^{20}\,5^{9}\,7^{6}\,11^{2}\,13^{3} \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 41 \cdot 47 \cdot 59 \cdot 71 \\ &= 808\,017\,424\,794\,512\,875\,886\,459\,904\,961\,710\,757\,005\,754\,368\,000\,000\,000, \end{aligned}$$

whose number of divisors is $\tau(|\mathbb{M}|) = 47 \cdot 21 \cdot 10 \cdot 7 \cdot 3 \cdot 4 \cdot 2^9 = 424\,488\,960$ and whose divisor sum is the 182-bit integer

$$\sigma_1(|\mathbb{M}|) = 5\,640\,122\,123\,081\,716\,028\,404\,428\,880\,410\,995\,850\,538\,621\,945\,774\,080\,000.$$

The question the host development answers is whether two *divisors* of $|\mathbb{M}|$ can be amicable.

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $M_{11}$ | 13 | $M_{12}$ | 17 | $J_1$ | 18 | $M_{22}$ | 19 | $J_2$ | 20 | $M_{23}$ | 24 | $HS$ | 26 |
| $J_3$ | 26 | $M_{24}$ | 28 | $McL$ | 30 | $He$ | 32 | $Ru$ | 38 | $Suz$ | 39 | $ON$ | 39 |
| $Co_3$ | 39 | $Co_2$ | 46 | $Fi_{22}$ | 46 | $HN$ | 48 | $Ly$ | 56 | $Th$ | 57 | $Fi_{23}$ | 62 |
| $Co_1$ | 62 | $J_4$ | 67 | $Fi'_{24}$ | 81 | $B$ | 112 | $\mathbb{M}$ | 180 | $\sigma_1(\mathbb{M})$ | 182 | | |

Table 1: Sizes in bits of the sporadic group orders (`sporadic_order_bitSizes`, `monster_bitSizes`). One group serves all of them.

**Theorem 2.1** (`MonsterDivisorAmicable.monster_divisor_amicable_search`)**.** *If $m \mid |\mathbb{M}|$, $n \mid |\mathbb{M}|$, $m < n$, $m \le 10^7$ and $(m, n)$ is amicable, then*

$$(m, n) \in \{(220, 284),\ (67095, 71145),\ (522405, 525915)\}.$$

*All three pairs do occur, with $\sigma_1$ equal to* 504*,* 138 240 *and* 1 048 320 *respectively, and the last two are odd.*

The pairs factor as $220 = 2^2 \cdot 5 \cdot 11$, $284 = 2^2 \cdot 71$, $67095 = 3^3 \cdot 5 \cdot 7 \cdot 71$, $71145 = 3^3 \cdot 5 \cdot 17 \cdot 31$, $522405 = 3^2 \cdot 5 \cdot 13 \cdot 19 \cdot 47$ and $525915 = 3^2 \cdot 5 \cdot 13 \cdot 29 \cdot 31$: every prime that occurs is one of the fifteen supersingular primes dividing $|\mathbb{M}|$, and both of the first two pairs need 71, the largest of them. Running the same sweep across all 26 sporadic groups with the smaller member bounded by $10^6$ leaves exactly one further group with an amicable pair in its divisor lattice:

**Theorem 2.2** (`SporadicDivisorAmicable.J4_divisor_amicable_search`)**.** $1184 = 2^5 \cdot 37$ *and* $1210 = 2 \cdot 5 \cdot 11^2$ *divide $|J_4|$ and satisfy $\sigma_1(1184) = \sigma_1(1210) = 1184 + 1210 = 2394$; below $10^6$ they are the only amicable pair of divisors of $|J_4|$, and the other* 24 *sporadic groups have none.*

These are the statements our protocol certifies without revealing the numbers. Note that no new mathematics enters: $(220, 284)$ is the pair known to the Pythagoreans and $(1184, 1210)$ is Paganini's; the content of Theorems 2.1 and 2.2 is the *completeness* of the search inside a specific divisor lattice, obtained by kernel computation over the exponent vectors of the fifteen supersingular primes.

### 2.2 The atlas of objects

The host development collects 1766 labelled integers into one table (`PrimeFiber.Atlas.atlas`): the 26 sporadic orders, their $\sigma_1$ and $\tau$ values, the aliquot sums the search compared, maximal-subgroup orders, the Monster's character degrees, folded McKay–Thompson coefficients, and the prime powers $p^e$ together with the partial sums $1 + p + \cdots + p^e$ out of which each $\sigma_1$ was assembled. Every one of them becomes a secret of the protocol. Their *sizes* — bit lengths — run from a couple of bits to

$$\texttt{bitSize}\,|\mathbb{M}| = 180, \qquad \texttt{bitSize}\,\sigma_1(|\mathbb{M}|) = 182,$$

and 182 bits is proved to bound the whole atlas (`atlas_bitSize_le`). The size ladder of the group orders is recorded in Table 1.

## 3 The cryptographic layer: standard components, reproduced

Nothing in this section is new; we record it only to fix notation and to name the Lean declarations that will be quoted later. Throughout, $q$ is a prime, $G$ is a module over $\mathbb{F}_q = \mathbb{Z}/q$ written additively, and $g, h \in G$.

### 3.1 Schnorr's proof of possession

**Definition 3.1.** The public key of a secret $x \in \mathbb{F}_q$ is $X = x \cdot g$. A transcript is a triple $(A, c, s) \in G \times \mathbb{F}_q \times \mathbb{F}_q$; it is *accepting* for $X$ when $s \cdot g = A + c \cdot X$. The honest prover with randomness $r$ answers a challenge $c$ with $(r \cdot g,\ c,\ r + cx)$; the simulator, given only $(g, X, c)$ and a random $s$, outputs $(s \cdot g - c \cdot X,\ c,\ s)$.

```
def key (g : G) (x : ZMod q) : G := x · g
def verify (g X : G) (t : Transcript G q) : Prop :=
  t.response · g = t.commitment + t.challenge · X
def prove (g : G) (x r c : ZMod q) : Transcript G q := ⟨r · g, c, r + c * ⟩x
def simulate (g X : G) (c s : ZMod q) : Transcript G q := ⟨s · g - c · X, c, ⟩s
def extract (₁t ₂t : Transcript G q) : ZMod q :=
  (₁t.response - ₂t.response) * (₁t.challenge - ₂t.challenge)⁻¹
```

**Theorem 3.2** (Completeness, special soundness, HVZK; `ZKPoP.Core`)**.** *For all $g, x, r, c$ the honest transcript is accepting (`verify_prove`). If two accepting transcripts share their first message and have different challenges, then `extract` returns a discrete logarithm of $X$ (`key_extract`); if $x \mapsto x \cdot g$ is injective it returns $x$ itself (`extract_eq_secret`). For each challenge $c$,*

$$\{t : \text{accepting for } X,\ t.\text{challenge} = c\} \;=\; \text{range}\big(\textit{simulate}\ g\ X\ c\big) \;=\; \text{range}\big(r \mapsto \textit{prove}\ g\ x\ r\ c\big),$$

*the first equality being `accepting_eq_simulated` and the second `honest_range_eq_simulated_range`.*

The two set equalities are the sharpest finitary form of honest-verifier zero knowledge: the honest prover's transcripts and the simulator's transcripts are not merely identically distributed, they are literally the same set, and the simulator reads only public data.

### 3.2 Pedersen commitments and homomorphic checks

$\text{ped}(x, r) = x \cdot g + r \cdot h$ is perfectly hiding when $h = k \cdot g$ with $k \neq 0$ (`ped_hiding`), and a collision yields $\log_g h$ (`ped_binding_extracts_dlog`): binding rests on the discrete-logarithm problem and on nothing else. Keys are additively homomorphic, `key_add`, and

$$\texttt{key\_injective\_iff}: \qquad x \cdot g + y \cdot g = s \cdot g \iff x + y = s,$$

which is the entire mechanism by which a verifier will check the amicable relation in §5 without seeing its terms.

### 3.3 OR-composition and range proofs

`ZKPoP.Range` reproduces the CDS OR-composition of two Schnorr proofs (`orProveLeft`, `orProveRight`, `or_special_soundness`, `accepting_or_eq_simulated`) and applies it in the standard way to prove that a Pedersen commitment hides a bit; `bit_sound_or_dlog` states the disjunction honestly — either the committed value is a bit, or the prover knows $\log_g h$. Bit commitments recompose, $\sum_i 2^i C_i = \text{ped}(\sum_i 2^i b_i,\ \sum_i 2^i r_i)$ (`commitSum_eq`), so certifying $k$ bits certifies the range (`bitsValue_lt_two_pow`, `range_sound`). This is how an object's *size* — its bit length — is proved without revealing the object.

### 3.4 The group, from a theorem instead of a ceremony

The concrete instance takes

$$q \;=\; 2^{521} - 1,$$

the Mersenne prime, and $G = \mathbb{Z}/q$ with generator 1. Primality is not assumed and not imported from a table: `ZKPoP.Objects.zkPrime_prime` runs the Lucas–Lehmer test inside the kernel.

```
theorem zkPrime_prime : Nat.Prime zkPrime := by
  rw [zkPrime_eq_mersenne]
  apply lucas_lehmer_sufficiency _ (by norm_num)
  norm_num
```

Since $2^{182} < q$ (`two_pow_182_lt_zkPrime`) every atlas object is a legitimate secret (`atlas_value_lt_zkPrime`), distinct objects have distinct public keys (`pubKey_injOn_atlas`), and a public key determines its object (`val_pubKey`). One group therefore serves a two-bit prime power and a 54-digit divisor sum alike, and a transcript has the same shape whatever it speaks about.

# 4 Self-hosting

## 4.1 The definition

**Definition 4.1** (Self-hosted ZKPoP)**.** Let $\mathcal{K}$ be a proof-checking kernel. A zero-knowledge proof of possession is *self-hosted in* $\mathcal{K}$ when all five of the following are theorems or executable definitions of one and the same $\mathcal{K}$-development:

**(S1) Statement.** The relation certified in zero knowledge is a proved theorem of the development, not an external assumption.

**(S2) Parameters.** The group is exhibited together with a proof of the arithmetic facts that make it a group of the claimed order — in particular, primality of the modulus is proved, not assumed, so there is no trusted setup.

**(S3) Protocol.** Completeness, special soundness and (honest-verifier) zero knowledge are theorems of the development, with no pen-and-paper step.

**(S4) Verifier.** The verifier is an executable function of the kernel's own language, and a theorem states that it decides the specification.

**(S5) Reflexivity.** The system's own description — its parameters, and a digest of the verifier's code — is a family of objects of the system, on which (S3) applies verbatim.

Conditions (S1)–(S3) say that the three trusted layers of a conventional deployment have been absorbed into the kernel; (S4) closes the implementation gap; (S5) is the property that justifies the word *self-hosted*, in the compiler's sense: the system is expressive enough to take itself as input.

## 4.2 The verifier as an executable function

```
def verifyB (X : Grp) (t : Transcript Grp zkPrime) : Bool :=
  decide (t.response · gen = t.commitment + t.challenge · X)

theorem verifyB_iff (X : Grp) (t : Transcript Grp zkPrime) :
    verifyB X t = true ↔ verify gen X t
```

`verifyB` is a total, kernel-evaluable Boolean function — the verifier one would actually run — and `verifyB_iff` proves it equivalent to the propositional specification of §3. Completeness in executable form, `verifyB_prove`, then says: for every natural number $n$ and every $r, c$, `verifyB (pubKey n) (prove gen (secret n) r c) = true`. This is (S4).

## 4.3 Possession of an arbitrary integer

The three protocol properties are restated once, for an arbitrary integer secret rather than only for atlas entries; each proof is a one-liner delegating to §3.

```
theorem zkpop_complete (n : ℕ) (r c : Grp) :
    verify gen (pubKey n) (prove gen (secret n) r c)

theorem zkpop_extract {n : ℕ} (hn : n < zkPrime) {₁t ₂t : Transcript Grp zkPrime}
    (hA : ₁t.commitment = ₂t.commitment) (hc : ₁t.challenge ≠ ₂t.challenge)
    (₁h : verify gen (pubKey n) ₁t) (₂h : verify gen (pubKey n) ₂t) :
    (extract ₁t ₂t).val = n

theorem zkpop_zero_knowledge (n : ℕ) (c : Grp) :
    {t : Transcript Grp zkPrime | verify gen (pubKey n) t ∧ t.challenge = c} =
      Set.range (simulate gen (pubKey n) c)
```

Special soundness returns the secret *as a natural number*, not merely as a field element: acceptance certifies possession of that specific integer. This is (S3).

### 4.4 Reflexivity: the system as one of its own objects

Two self-descriptions are made into objects. The first is numeric:

```
def sysParams : List ℕ := [521, 182, 1766, 26, 3, 1]
theorem sysParams_lt_zkPrime : ∀ n ∈ sysParams, n < zkPrime
```

— the bit length of the modulus, the common object width, the size of the atlas, the number of sporadic groups, and the numbers of amicable pairs found among the divisors of $|\mathbb{M}|$ and of $|J_4|$. Each is a legitimate secret, so `selfhost_params_complete`, `selfhost_params_extract` and `selfhost_params_zero_knowledge` apply: the system proves possession of its own description, and the proof reveals nothing beyond the public keys.

The second is textual. A Horner digest modulo $q$ turns a string into a field element:

```
def digest (s : String) : ℕ :=
  s.toUTF8.toList.foldl (fun a b => (a * 256 + b.toNat) % zkPrime) 0

theorem digest_lt_zkPrime (s : String) : digest s < zkPrime

def verifierCode : String :=
  "verifyB X t = decide (t.response · gen = t.commitment + t.challenge · X)"

def selfCode : ℕ := digest verifierCode
```

No cryptographic property of `digest` is claimed or needed — it is an encoding, not a hash — and `digest_lt_zkPrime` is the only thing required of it: that its output is a legitimate secret. Then `selfhost_code_complete`, `selfhost_code_extract` and `selfhost_code_zero_knowledge` say that the system proves possession of the code of its own verifier, that two accepting runs extract that code exactly, and that the runs are simulatable from public data. This is (S5).

### 4.5 The package

**Theorem 4.2** (`ZKPoP.SelfHost.self_hosted_zk_system`)**.** *The following hold simultaneously, in one Lean development, with no* `sorry` *and no non-standard axiom:*

1. $2^{521} - 1$ *is prime* (S2)*;*
2. $\forall X, t$, `verifyB X t = true` $\iff$ `verify gen X t` (S4)*;*
3. $\forall n, r, c$, `verifyB (pubKey n) (prove gen (secret n) r c) = true`*;*
4. $\forall n < q$*, two accepting transcripts with equal first message and distinct challenges extract* $n$ (S3)*;*
5. $\forall n, c$*, the accepting transcripts at challenge* $c$ *are exactly the simulator's outputs* (S3)*;*

6. *every element of* `sysParams` *and* `selfCode` *is a legitimate secret* (S5)*;*

7. `pubKey 220 + pubKey 284 = pubKey 504` *together with* $\sigma_1(220) = \sigma_1(284) = 504$ (S1)*.*

Item (7) is the hinge: the relation the verifier checks on public keys is the same relation that Theorem 2.1 proves about integers, and both facts are conjuncts of one theorem.

### 4.6 What self-hosting is not: the Gödelian boundary

It is worth being pedantic here, because the phrase invites over-reading. Definition 4.1 is a statement about *where the components come from*, not a statement of self-justification. In particular:

- We do not prove, and by Gödel's second incompleteness theorem cannot prove inside the same system, that the kernel is consistent. If the kernel is unsound, everything here is worthless — self-hosting does not and cannot repair that.
- `selfCode` is a digest of a *string* that transcribes the verifier's defining equation; it is not a Gödel number of the proof term, and no fixed-point or diagonalisation argument is involved. The reflexivity in (S5) is of the compiler kind (the system accepts its own description as input), not of the provability-logic kind.
- Conversely, the compiler analogy is exact in the place where it matters: a self-hosting compiler does not prove itself correct either, but it does eliminate an entire class of version-skew and translation errors between the specification and the artefact. (S4) does precisely that for the verifier.

## 5 The application: an amicable pair, in zero knowledge

### 5.1 The protocol run

Fix the pair $(m, n) = (1184, 1210)$ of divisors of $|J_4|$ and let $s = \sigma_1(m) = \sigma_1(n) = 2394$. The prover publishes $X_m = m \cdot g$ and $X_n = n \cdot g$ and proceeds:

1. **Possession.** Two Schnorr runs, one per number, with fresh randomness; the verifier runs `verifyB` on each. Completeness is `zkpop_complete`; a rewinding extractor recovers $m$ and $n$ exactly, by `zkpop_extract`.
2. **Relation.** The verifier computes $X_m + X_n$ and compares it with the public key of $s$. By `hidden_sum_iff` this equality holds if and only if $m+n = s$, provided both sides are below $q$ — which `lt_zkPrime_of_lt_two_pow` discharges.
3. **Arithmetic.** That $s$ really is the common divisor sum is not something the prover asserts: $\sigma_1(1184) = \sigma_1(1210) = 2394$ is recomputed by the kernel (`sigma_1184`, `sigma_1210`).
4. **Zero knowledge.** By `zkpop_zero_knowledge` each transcript lies in the range of the simulator, which sees only $(g, X, c)$. The verifier ends convinced that the prover holds two numbers whose sum is 2394 and whose divisor sums are both 2394, and can compute nothing about the numbers that it could not have computed from the public keys alone.

**Theorem 5.1** (`ZKPoP.Objects.amicable_J4_pair_checked_on_commitments`)**.** `pubKey 1184 + pubKey 1210 = pubKey 2394`*, and* $\sigma_1(1184) = \sigma_1(1210) = 2394$*, and both numbers are provable in possession for every choice of randomness and challenge.*

| Ingredient | Source | New here? |
|---|---|---|
| $\Sigma$-protocol for discrete log | Schnorr | no |
| Perfectly hiding commitment | Pedersen | no |
| OR-composition | Cramer–Damgård–Schoenmakers | no |
| Bit-decomposition range proof | folklore | no |
| Non-interactivity | Fiat–Shamir (discussed, §7) | no |
| Prime modulus $2^{521} - 1$ | Lucas–Lehmer | no |
| Multiplicativity of $\sigma_1$ | classical, in Mathlib | no |
| Amicable pairs $(220, 284)$, $(1184, 1210)$ | classical | no |
| Completeness of the divisor sweep | host development | no (prior work) |
| Definition of self-hosted ZKPoP | Definition 4.1 | **yes** |
| Executable verifier + adequacy | `verifyB`, `verifyB_iff` | **yes** |
| Self-description objects | `sysParams`, `selfCode` | **yes** |
| Packaging theorem | `self_hosted_zk_system` | **yes** |

Table 2: What is reused and what is new. New formal content: `RequestProject/ZKPoP/SelfHost.lean`, 236 lines, on top of 819 lines of previously verified protocol code.

**Theorem 5.2** (`ZKPoP.SelfHost.monster_amicable_pairs_checked_on_commitments`)**.** *The same holds for all three amicable pairs of divisors of* $|\mathbb{M}|$*:*

$$\begin{aligned} X_{220} + X_{284} &= X_{504}, & \sigma_1(220) &= \sigma_1(284) = 504, \\ X_{67095} + X_{71145} &= X_{138240}, & \sigma_1(67095) &= \sigma_1(71145) = 138\,240, \\ X_{522405} + X_{525915} &= X_{1048320}, & \sigma_1(522405) &= \sigma_1(525915) = 1\,048\,320, \end{aligned}$$

*and* `verifyB` *accepts the honest proof of possession of each of the six numbers, for every randomness and challenge.*

### 5.2 Hiding the size as well as the value

A commitment to an object is uniformly distributed over the group (`pedersen_perfectly_hiding`): identically so for a two-bit prime power and for the 182-bit $\sigma_1(|\mathbb{M}|)$, so not even the size leaks. When the prover *wants* to reveal the size and nothing else — "the hidden divisor has at most $k$ bits" — the range machinery of §3 does it: every object recomposes from its bits (`atlas_bits_recompose`, `bits_recompose_own_size`) and the verifier's recomposition of the bit commitments is a commitment to the object (`atlas_size_proof_complete`, `size_proof_complete_own_size`). Size is thus a disclosable attribute rather than an unavoidable leak, which is what makes Table 1 a menu of statements a prover may choose to make.

### 5.3 Batching the whole atlas

`zkpop_atlas_batch` runs the AND-composition over all 1766 objects with one shared challenge and one transcript apiece; `batch_extract` extracts entrywise. A prover can therefore demonstrate possession of the entire arithmetic output of the host development in a single interaction.

## 6 The minimality ledger

Table 2 accounts for every ingredient. The rightmost column is the point of the paper: the "new" entries are one definition and one file of glue.

The same accounting applies to the *disclosure* of the protocol itself. Running the system on the pair $(m, n)$ discloses: the public keys $X_m$, $X_n$; the public key $X_s$ of their sum; and, if the

prover chooses, the bit lengths. It discloses nothing else — by Theorem 3.2 anything a verifier can compute from a transcript, it can compute from the public key by running `simulate`.

# 7 Trust ledger and limitations

**Axioms.** Every theorem cited here is checked with no `sorry`. The protocol theorems (`self_hosted_zk_system`'s protocol conjuncts, `selfhost_code_extract`, `amicable_J4_pair_checked_on_commitments`) depend only on `propext`, `Classical.choice` and `Quot.sound`. The conjuncts that quote the arithmetic sweep — Theorem 2.1 and the $\sigma_1$ values of the three Monster pairs — are established by compiled kernel evaluation and therefore additionally depend on `Lean.ofReduceBool` and `Lean.trustCompiler`, i.e. they trust the Lean compiler in addition to the kernel. We flag this rather than hide it: it is the one place where the arithmetic layer is checked by a larger trusted base than the cryptographic layer.

**The representation.** $G = \mathbb{Z}/(2^{521} - 1)$ is a genuine cyclic group of prime order, and every claim above holds in it unconditionally — completeness, special soundness, perfect HVZK, perfect hiding, recomposition, "binding implies discrete log". But discrete logarithms in *this* representation are trivial (division), so the instance is a faithful arithmetic model of the protocol, not a deployment. Deployment means reading the same `ZKPoP.Core` theorems — which are stated for an arbitrary $\mathbb{F}_q$-module — in a group where the discrete-logarithm problem is believed hard. Nothing in §4 changes under that substitution except the concrete numbers, and the statements that genuinely need binding (`range_sound`, `bit_sound_or_dlog`) are already stated abstractly, with base independence as an explicit hypothesis or with "…or the prover knows the discrete logarithm" in the conclusion.

**Honest-verifier only.** What is proved is honest-verifier zero knowledge, in its strongest set-theoretic form. Full (malicious-verifier) zero knowledge and the Fiat–Shamir transform are standard next steps and are *not* formalised here; in particular no random oracle is modelled, and the §5 protocol is interactive.

**Scope of the arithmetic.** Theorem 2.1 is complete below $10^7$ for the Monster and below $10^6$ across the sporadic groups; it is not a statement about all 424 488 960 divisors. The zero-knowledge layer inherits exactly this scope — it certifies possession of numbers satisfying the amicable relation, and the completeness of the enumeration is a separate, non-hidden theorem.

**Extraction is rewinding-based.** Special soundness is the two-transcript statement; turning it into a proof of knowledge in the standard sense requires the usual rewinding argument, which we quote rather than formalise.

# 8 Conclusion

Taking a machine-checked arithmetic result and a machine-checked Σ-protocol and insisting that they live in one kernel costs, in this instance, 236 lines and one definition. What it buys is that the four layers usually trusted separately — statement, parameters, protocol proof, verifier code — are all discharged by the same checker, and that the system is expressive enough to run on its own description. The concrete instance certifies, in zero knowledge, that its prover holds two divisors of the Monster's order that are amicable; the verifier learns their sum's public key and nothing else; and the fact that they are amicable, the fact that they are the only such divisors below $10^7$, the primality of the group's modulus, the soundness of the protocol and the correctness of the verifier that checks it are all theorems of the same development.

# A Index of formal declarations

| Declaration | File | Role |
|---|---|---|
| verify_prove | ZKPoP/Core.lean | completeness |
| key_extract, extract_eq_secret | ZKPoP/Core.lean | special soundness |
| accepting_eq_simulated | ZKPoP/Core.lean | HVZK, set form |
| honest_range_eq_simulated_range | ZKPoP/Core.lean | HVZK, distribution form |
| ped_hiding | ZKPoP/Core.lean | perfect hiding |
| ped_binding_extracts_dlog | ZKPoP/Core.lean | binding ⇒ dlog |
| key_add, key_injective_iff | ZKPoP/Core.lean | homomorphic check |
| batch_complete, batch_extract | ZKPoP/Core.lean | AND-composition |
| or_special_soundness | ZKPoP/Range.lean | OR-composition |
| accepting_or_eq_simulated | ZKPoP/Range.lean | OR simulation |
| bit_sound_or_dlog | ZKPoP/Range.lean | bit soundness |
| commitSum_eq, range_sound | ZKPoP/Range.lean | range proof |
| zkPrime_prime | ZKPoP/Objects.lean | group parameters (S2) |
| atlas_bitSize_le, bitSize_spec | ZKPoP/Objects.lean | sizes |
| pubKey_injOn_atlas, val_pubKey | ZKPoP/Objects.lean | keys determine objects |
| zkpop_atlas_complete/extract/... | ZKPoP/Objects.lean | protocol on the atlas |
| amicable_J4_pair_ checked_on_commitments | ZKPoP/Objects.lean | application |
| verifyB, verifyB_iff | ZKPoP/SelfHost.lean | executable verifier (S4) |
| sysParams_lt_zkPrime | ZKPoP/SelfHost.lean | self-description (S5) |
| digest_lt_zkPrime, selfCode | ZKPoP/SelfHost.lean | code as object (S5) |
| monster_amicable_pairs_ checked_on_commitments | ZKPoP/SelfHost.lean | application (S1) |
| self_hosted_zk_system | ZKPoP/SelfHost.lean | Theorem 4.2 |
| monster_divisor_amicable_search | MonsterDivisorAmicable.lean | Theorem 2.1 |
| J4_divisor_amicable_search | SporadicDivisorAmicable.lean | Theorem 2.2 |